

Fiche TD-TP n°7 : Les listes et les dictionnaires

Les Listes

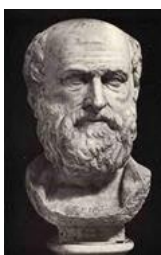
Exercice 1

Remplir une liste d'entiers **Liste<entier>** par 50 entiers tirés au sort entre 1 et 100.

En utilisant les primitives de la structure de donnée Liste<T> :

- Afficher cette liste en utilisant la structure **Pour chaque**,
- Afficher cette liste à l'envers,
- Trier cette liste par ordre croissant et réafficher la liste,
- Trier cette par ordre décroissant et réafficher la liste.

Exercice 2



Eratosthène – Astronome (Décès en 194 av. J.-C)

On veut créer la liste des nombres premiers inférieurs à une borne saisie par l'utilisateur en utilisant le **crible d'Eratosthène**. Voici le principe avec un exemple où l'utilisateur a saisi 20 :

Etape 1 : On crée et remplit une liste avec tous les nombres de 2 à 20

2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

Etape 2 : On prend le 1^{er} nombre de la liste (ici 2) et on retire de la liste tous les multiples de 2

2 3 5 7 9 11 13 15 17 19

Etape 3 : On prend le 2^{ème} nombre de la liste (ici 3) et on retire de la liste tous les multiples de 3

2 3 5 7 11 13 17 19

Etape n : et ainsi de suite jusqu'à la fin de la liste...

A la fin afficher la liste, il ne reste que les nombres premiers.

Exercice 3

- a) Créer une fonction **fusionner()** qui reçoit en paramètres 2 listes **lst1** et **lst2** et qui retourne une liste contenant la fusion triée de **lst1** et **lst2**. Par exemple :

lst1 : 2 3 4 9

lst2 : 1 5 6 7 8

- b) Créer l'algorithme principal qui appelle cette fonction avec l'exemple précédent et qui affiche la nouvelle liste créée.

Contraintes : **interdit d'utiliser la méthode Trier()**
 interdit de modifier lst1 ni lst2

L'appel **fusionner(lst1, lst2)** devra renvoyer la liste : 1 2 3 4 5 6 7 8 9

Les dictionnaires

Exercice 4

a) Dictionnaire <chaine, entier>

- Demander à l'utilisateur de saisir une année,
- Créer le dictionnaire qui stocke le nombre de jours des mois de l'année saisie

Par exemple si l'utilisateur saisit 2026, étant donnée qu'il ne s'agit pas d'une année bissextile voici le dictionnaire **lesJours** à créer :

clé	janvier	février	mars	avril	mai	juin	juillet	août	septembre	octobre	novembre	décembre
valeur	31	28	31	30	31	30	31	31	30	31	30	31

- Afficher le contenu du dictionnaire sous la forme :

Année : 2026 :

janvier : 31 jours

février : 28 jours

...

décembre : 31 jours

Total : 365 jours

b) Dictionnaire <entier, Joueur>

- Créer la structure **Joueur** suivante :
avec un constructeur

Structure Joueur

nom : chaine

prenom : chaine

FinStructure

- Créer 3 joueurs dans les variables j1, j2 et j3.

On veut créer un dictionnaire **lesJoueurs** où l'on stockera le numéro du joueur en clé et le joueur en valeur :

- Remplir le dictionnaire **lesJoueurs** avec les 3 joueurs j1, j2 et j3

clé	55956	56871	57213
	nom Einstein	nom TURING	nom LOVELACE
valeur	premier Albert	premier Alan	premier Ada

- Afficher le tableau des joueurs sous cette forme :

Joueur n° 55956 : Albert Einstein

Joueur n° 56871 : Alan TURING

Joueur n° 57213 : Ada LOVELACE

c) Dictionnaire <Joueur, entier>

On veut maintenant créer un dictionnaire **lesScores** où l'on stockera pour chaque joueur le score obtenu lors d'une session de jeu :

clé	nom Einstein	nom TURING	nom LOVELACE
	premier Albert	premier Alan	premier Ada
valeur	323	188	413

- Créer et remplir ce dictionnaire avec les scores de j1, j2 et j3.
- Afficher ce tableau sous la forme :

Joueur : Albert Einstein, score = 323

Joueur : Alan TURING, score = 188

Joueur : Ada LOVELACE, score = 413

Gagnant : Ada LOVELACE

Tableaux + Listes + Dictionnaires

Exercice 5 Liaisons aériennes (ou lésions ARN)

Une compagnie aérienne possède plusieurs avions permettant d'effectuer des vols entre Strasbourg et plusieurs villes. L'ensemble de ces destinations est stocké dans une liste **destinations**. Par exemple : **destinations** : <Monte-Carlo, Paris, Rennes>

Cette compagnie emploie plusieurs pilotes pour effectuer ses vols. Leur nom est stocké dans une liste **pilotes**. Par exemple : **pilotes** : <Dany, Laurent, Emilie, Alex>

Le nombre de liaisons effectuées par les pilotes pour chaque destination est enregistré dans un tableau à 2 dimensions **liaisons**. Les lignes correspondent aux pilotes et les colonnes aux destinations. A la fin de l'année, un récapitulatif des liaisons est calculé pour chaque pilote et chaque destination. Par exemple :

liaisons :

5	10	3
2	14	5
4	7	6
8	0	12

Donc cette année Dany a effectué 10 liaisons vers Paris et 3 liaisons vers Rennes.

Missions :

A vous d'analyser ce qu'il faut passer en paramètre dans les fonctions !

1. Écrire la fonction **calculVol()** qui calcule le nombre total de vols effectués.
2. Écrire la fonction **recapTotal()** de création d'un dictionnaire ayant pour clé le nom de chaque pilote et pour valeur son nombre total de vols effectués durant l'année

Pour cette année, le résultat attendu sera :

<Dany,18>
<Laurent,21>
<Emilie,17>
<Alex,20>

3. Écrire la procédure d'affichage de ce dictionnaire.

4. Écrire la fonction **villeDetails()** de création d'un dictionnaire, ayant pour clé une destination et pour valeur la liste des pilotes ayant effectué au moins un vol vers cette destination.

Pour cette année, le résultat attendu sera :

```
<Monte-Carlo,<Dany, Laurent, Emilie, Alex>>  
<Paris, <Dany, Laurent, Emilie>>  
<Rennes,<Dany, Laurent, Emilie, Alex>>
```

5. Écrire la procédure d'affichage de ce dictionnaire.

6. Écrire l'algorithme principal qui :

- Affiche le nombre total de vols
- Affiche pour chaque pilote son nombre de vols
- Affiche pour chaque destination la liste des pilotes

Exercice 5 Gestion des vols commerciaux

Impressionné par le code écrit pour l'exercice précédent, la compagnie aérienne vous confie maintenant la création d'une application pour calculer le prix des billets pour les passagers et le chiffre d'affaires d'un vol.

Les informations d'un vol sont actuellement stockées dans 3 fichiers texte :

- Le fichier Cxxx.txt, par exemple **C707.txt** qui contient les différentes classes du vol 707, ce fichier est basé sur la structure suivante :

```
Structure Classe
    code : chaîne
    nom : chaîne
    prix : réel
FinStructure
```

- le fichier Pxxx.txt, par exemple **P707.txt** qui contient les passagers du vol 707, ce fichier est basé sur la structure suivante :

```
Structure Passager
    numero : entier
    nom : chaîne
    prenom : chaîne
    codeClasse : chaîne
FinStructure
```

Important : Ce fichier est trié par numéro de billet

- le fichier Bxxx.txt, par exemple **B707.txt** qui contient les bagages des passagers du vol 707, ce fichier est basé sur la structure suivante :

```
Structure Bagage
    numero : entier
    poids : entier
    numBillet : entier
FinStructure
```

Important : Ce fichier est trié par numéro de billet

Pour l'ensemble de ces 3 fichiers le délimiteur utilisé pour séparer les informations est le ';'.

Après avoir demandé à l'utilisateur de saisir le numéro du vol, l'application devra proposer le menu suivant :

- 1-Prix du billet d'un passager
- 2-Liste des passagers
- 3-Chiffre d'affaires du vol
- 4-Quitter

Remarques :

- Pour le choix 1 on saisit le numéro de passager (pour aller plus loin on pourrait aussi saisir le nom, même partiel et choisir le passager dans une liste).
- Pour le choix 2 la liste doit respecter cet exemple :

Vol 707				
Numéro	Passager	Classe	Nombre bagages	Poids cumulé
18	Emma Kim	Première classe	1	9 Kg

Missions :

- Créer une fonction **ChargerListeClasse()** qui charge une liste de <Classe> à partir du fichier Cxxx.txt dont le numéro de vol est reçu en paramètre.
- Créer une fonction **ChargerListePassager()** qui charge une liste de <Passager> à partir du fichier Pxxx.txt dont le numéro de vol est reçu en paramètre.
- Créer une fonction **ChargerListeBagage()** qui charge une liste de <Bagage> à partir du fichier Bxxx.txt dont le numéro de vol est reçu en paramètre.
- Créer une procédure **AfficherPassagers()** qui affiche la liste des passagers au format attendu
- Créer une fonction **CalculerPrix()** qui reçoit en paramètre un numéro de passager et qui calcule le prix de son billet. La règle de gestion est la suivante :

Le prix du billet est déterminé par le prix de la classe choisie plus éventuellement une surtaxe en fonction du poids cumulé des bagages (un passager peut avoir plusieurs bagages).

Jusqu'à 20 kg : pas de surtaxe
 21-30 kg : 10% de surtaxe
 31-40 kg : 20% de surtaxe
 41 et plus : 30 % de surtaxe

- Le programme principal devra gérer la saisie du numéro de vol et le menu.

Vous pouvez créer d'autres fonctions/procédures au besoin.